Universidad Nacional Autó
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 1.4

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 14/09/2026

Uso de librerías para arreglos y matrices en Numpy

La librería numérica de Python más conocida por su nombre simplificado es Numpy, una de las más relevantes dentro de este lenguaje, convirtiéndose en un estándar dentro de la comunidad de programación científica y en casi todos los desarrollos relacionados a la ciencia de datos y la inteligencia artificial.

Según McKinney (2017), algunos de los elementos claves de numpy son:

La implementación de un eficiente arreglo multidimensional (ndarray), el cual está orientado a optimizar la ejecución de operaciones aritméticas. Contiene varias funciones matemáticas diseñadas para realizar operaciones con arreglos de datos significativamente grandes de forma rápida y mediante la escritura de un código más limpio. Posee un conjunto de herramientas para la lectura y escritura de datos en almacenamientos externos utilizando ficheros mapeados en memoria. Está optimizada

para resolver problemas de álgebra lineal, la generación de valores aleatorios y realizar operaciones en el dominio de la frecuencia. Implementa una interfaz de punto de acceso (API) en lenguaje C, que permite conectarla con librerías escritas en otros lenguajes de programación. Numpy es muy utilizado para servir de interfaz entre antiguos programas escritos en C, C++ o Fortran y las nuevas formas de programación orientadas a arreglos; esta facilidad incrementa la vida útil de muchas aplicaciones científicas que de otra forma ya estarían descontinuadas. Es importante señalar que numpy en sí, no provee de muchas funciones para generar modelos matemáticos complejos, pero la capacidad de trabajo con arreglos y su forma específica de programación, se combinan de manera excelente con otras librerías científicas, creando una simbiosis imposible de superar.

Según McKinney (2017), para la mayoría de las aplicaciones de análisis de datos, las principales funcionalidades que esta librería ofrece son:

• Realización de complejas operaciones con matrices, de forma rápida y enfocadas en el intercambio, limpieza de datos, generación de subconjuntos, filtrado de valores, transformaciones entre tipos de datos o cualquier otro tipo de cálculo.

• Uso de algoritmos básicos para realizar operaciones dentro de las matrices; por ejemplo, la ordenación de elementos de forma individual o de conjuntos.

• Eficiente descripción de estadísticas, agregación o resumen de datos.

• Facilidad de manipular datos relacionales y realizar las operaciones unión o fusión de estos en nuevos conjuntos de datos.

• Utilizar elementos de lógica condicional como expresiones dentro de los arreglos, en lugar de utilizar bucles u otras estructuras de control.

• Manipulaciones de datos grupales (agregación, transformación o aplicación de funciones).

Una de las razones por las que numpy es tan importante para la computación numérica en Python, es porque está diseñado especialmente para trabajar con grandes arreglos de datos. Algunas de las consideraciones que se tuvieron a la hora de diseñar esta librería fueron (McKinney, 2017):

Almacenar los datos de forma interna en bloques contiguos de memoria, haciéndolo de forma diferente a como Python manejaría de forma nativa datos con iguales características, esto es posible por la forma en que opera el algoritmo de bajo nivel escrito en C y por supuesto agregando la ventaja de que se consume una menor cantidad de recursos de memoria.

Diseñar todas las operaciones computacionales con arreglos que realiza la librería, utilizando la menor cantidad posible de ciclos o estructuras de control.

Todas las funcionalidades de esta librería se encuentran dentro del paquete numpy. La forma recomendada de incorporarlo dentro de nuestros proyectos

```
import numpy as np
```

Se puede utilizar cualquier prefijo para referirse a esta librería, pero es muy común encontrar en la bibliografía el término “np” asociado a la misma. En el desarrollo de este tema esa será la forma que utilizaremos para referenciarla.

Arreglos en Numpy

Una de las características principales de numpy es su arreglo n-dimensional, más conocido como “ndarray”, el cual es un contenedor rápido y flexible para cualquier conjunto de datos en Python. Un arreglo de numpy, nos permite realizar operaciones matemáticas a grandes bloques de datos, con la misma sencillez que se efectuaría con cada elemento de forma independiente.

Un “ndarray” es contenedor genérico multidimensional para datos homogéneos, lo cual significa que todos sus elementos tienen que ser del mismo tipo. Cada contenedor tiene una forma y un tipo de dato, esta información nos devuelve el tamaño y el tipo de valor que está alojando el arreglo durante su uso.

Un “ndarray” se crea de la siguiente forma

```
datos = [6, 4, 3.5, 8, 6, 9]
mi_primer_ndarray = np.array(datos)
```

Si quisiéramos verificar los atributos de dimensión y tipo de datos del arreglo, lo pudiéramos hacer como lo muestran

Comprobación de la característica de dimensión del arreglo.

```
mi_primer_ndarray.shape
```

```
(6,)
```

Comprobación del tipo de datos que conforma el arreglo.

```
mi_primer_ndarray.dtype
```

```
dtype('float64')
```

De la información anterior podemos concluir que nuestro arreglo es unidimensional, formado por una fila con 6 elementos, los cuales son del tipo de punto flotante de doble precisión (64bits).

Se muestran algunas funciones que incluye la librería y que nos permiten operar con los arreglos de diferentes formas:

Función	Descripción
Array	Convierte los datos de entrada (lista, tupla, arreglo o cualquier otra estructura de datos) en un *ndarray*. El tipo de datos puede ser inferido o indicado específicamente. Esta función realiza una copia de los datos de entrada.
Asarray	Convierte los datos de entrada en *ndarray*, pero no realiza la copia si detecta que la entrada ya es un *ndarray*.
Arange	Devuelve valores espaciados uniformemente dentro de un intervalo dado con el formato de un *ndarray* (Tabla 6).
Ones	Genera un arreglo de unos, con la forma y el tipo de datos indicados (Tabla 7).
Empty	Crea un arreglo con el tamaño y forma indicados, pero solamente reserva su espacio en memoria sin inicializar los valores (Tabla 8).
Full	Crea un arreglo con el tamaño y forma dados, además de inicializar los elementos con el valor indicado (Tabla 9).

```
#Uso de la función arange()
np.arange(10)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
#Uso de la función ones()
np.ones(10)
```

```
array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

```
#Uso de la función empty()
np.empty([3, 2])
```

```
array([[2.34330395e-319, 0.00000000e+000],
       [6.69944959e-310, 6.69944960e-310],
       [5.16665394e-317, 6.69944960e-310]])
```

```
#Uso de la función full()
np.full((2, 2), 5)
```

```
array([[5, 5],
       [5, 5]])
```

Tipos de datos para ndarray

Los tipos de datos para los “ndarray”, conocidos como “dtype”, son objetos especiales que contienen la información que el ndarray necesita interpretar para ajustar, de la forma más adecuada, el espacio asignado en memoria para un tipo de datos en particular. En la mayoría de las ocasiones se utilizan los tipos de datos inferidos por la librería, pero en función de la aplicación que se esté desarrollando, puede ser que definir cada uno de los elementos de forma independiente, sea la mejor opción (Hill, 2020).

En numpy los “dtype” coinciden con las nomenclaturas de otros lenguajes de bajo nivel como C o Fortran; ya que la librería realiza la manipulación de la memoria de forma similar a la ejecutada por estos otros lenguajes. En la Tabla 10 se muestran algunos tipos de “dtype”:

Tipo	Descripción
int8	Tipo entero, con o sin signo, representado con 8 bits de memoria.
int64	Tipo entero, con o sin signo, representado con 64 bits de memoria.
float32	Tipo número de punto flotante estándar, de precisión media, compatible con el float del lenguaje C, representado con 32 bits de memoria.
complex64	Número complejo, representado por dos float32, utiliza 64 bits de memoria.
bool	Tipo booleano, toma valores de Verdadero o Falso.
string	Cadena de tipo ASCII auto ajustada, de 1 byte por carácter en espacio de memoria.

En numpy disponemos del método astype(), el cual nos permite convertir o transformar datos numéricos de un tipo a otro. Esta función es muy útil cuando deseamos optimizar el uso de memoria, o queremos tener un control preciso de nuestros datos. En la Tabla 11 se muestra un ejemplo de su uso:

```
segundo_arreglo = np.array([1,2,3,4,5,6])
segundo_arreglo.dtype
```

```
dtype('int64')
```

```
segundo_arreglo_int8 = segundo_arreglo.astype(np.int8)
segundo_arreglo_int8.dtype
```

```
dtype('int8')
```

Operaciones aritméticas con arreglos

Según Hill (2020), los arreglos de numpy son muy versátiles al momento de realizar operaciones aritméticas porque nos ahorran tener que utilizar implementaciones complejas, llenas de ciclos y estructuras de control. Normalmente los usuarios

denominan a estas operaciones como “vectorización”, y las mismas se pueden aplicar entre arreglos de igual tamaño sin ninguna complicación. Se muestran algunos ejemplos de estas:

```
arr1 = np.array([[1,2,3],[4,5,6]])
arr1
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
arr2 = np.ones((2,3))
arr2
```

```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
arr1 + arr2
```

```
array([[2., 3., 4.],
       [5., 6., 7.]])
```

Indexación y rebanadas

Conociendo la forma en que numpy realiza la indexación podemos optar por implementar diferentes formas de seleccionar elementos dentro de los arreglos. Para tener una referencia visual nos podemos apoyar con la Figura 1, la cual muestra cómo se organizan los índices en una estructura multidimensional.

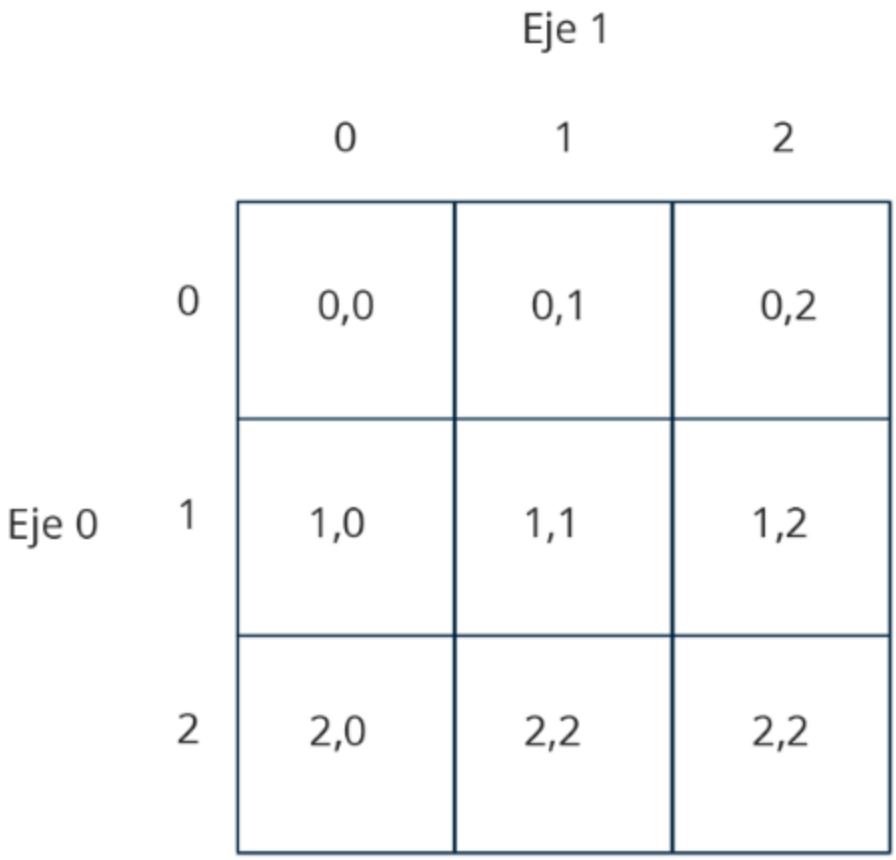


Figura 1. Índices en una estructura multidimensional.

Cuando trabajamos con "ndarray" de una sola dimensión, se opera de forma muy parecida al trabajo con listas. En la Tabla 12 podemos ver cómo seleccionar un elemento por su índice:

```
arr3 = np.arange(10)
arr3
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
#Selección de un elemento por su índice correspondiente.
arr3[6]
```

```
np.int64(6)
```

Podemos seleccionar un rango dentro del ndarray y realizar algunas operaciones directamente sobre este. En la Tabla 13, se muestra un ejemplo donde se le asigna el valor 10 a la selección de los elementos que van desde el índice 2 hasta el 5, utilizando de una sintaxis similar a la implementada con las listas.

```
#Selección de varios elementos.
arr3[2:6] = 10
arr3
```

```
array([ 0,  1, 10, 10, 10, 10,  6,  7,  8,  9])
```

De igual forma podemos utilizar rebanadas, para hacer selecciones más complejas. Supongamos que se desea extraer los elementos mostrados en la Figura 2, entonces la forma adecuada de hacerlo serían las mostradas

0	1	2	3	4	5
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	41	43	44	45
50	51	52	53	54	55

Figura 2. Arreglo de datos

```
# Creamos la matriz de 6x6 con los números correspondientes
# El truco aquí es usar arreglos que representen las decenas y unidades
filas = np.arange(0, 60, 10).reshape(6, 1)
columnas = np.arange(0, 6)
arr4 = filas + columnas

print(arr4)
```

```
[[ 0  1  2  3  4  5]
 [10 11 12 13 14 15]
 [20 21 22 23 24 25]
 [30 31 32 33 34 35]
 [40 41 42 43 44 45]
 [50 51 52 53 54 55]]
```

```
arr4[2::2,::2]
```

```
array([[20, 22, 24],
       [40, 42, 44]])
```

```
#Rebanada correspondiente a los elementos de color verde
arr4[:,2]
```

```
array([ 2, 12, 22, 32, 42, 52])
```

Indexado booleano e indexado elegante

En ocasiones es cómodo utilizar un arreglo booleano para definir los elementos a seleccionar en un arreglo, esta operación se puede realizar fácilmente de esta manera

```
arr3
```

```
array([ 0,  1, 10, 10, 10, 10,  6,  7,  8,  9])
```

```
#Indexado Booleano.
mis_indices = [True, False, True, False, False, True, False, True, False, False
arr3[mis_indices]
```

```
array([ 0, 10, 10,  7])
```

El indexado elegante es un término adoptado por numpy para describir la forma de utilizar como argumento una lista al momento de hacer una selección

```
arr3
```

```
array([ 0,  1, 10, 10, 10, 10,  6,  7,  8,  9])
```

```
mis_indices = [1, 5, -1]
arr3[mis_indices]
```

```
array([ 1, 10,  9])
```

Funciones universales y manejo de entradas/salidas

Según Hill (2020), una función universal o ufunc, es un tipo de función que puede ejecutar operaciones en cada uno de los elementos que conforman un ndarray. De esta forma podemos procesar de rápidamente todos los elementos del arreglo y desarrollar soluciones muy interesantes. Un ejemplo de ufunc puede ser tan simple como el cálculo de la raíz cuadrada o del exponencial de un elemento

```
arr5 = np.arange(8)
arr5
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

```
#Uso de la función universal sqrt()
np.sqrt(arr5)
```

```
array([0.        , 1.        , 1.41421356, 1.73205081, 2.        ,
       2.23606798, 2.44948974, 2.64575131])
```

```
#Uso de la función universal exp()
np.exp(arr5)
```

```
array([1.00000000e+00, 2.71828183e+00, 7.38905610e+00, 2.00855369e+01,
       5.45981500e+01, 1.48413159e+02, 4.03428793e+02, 1.09663316e+03])
```

Se muestran algunas de las funciones universales más comunes:

Función	Descripción
abs, fabs	Devuelve el valor absoluto de un elemento (soporta datos tipo entero, flotante o complejo).
sqrt	Calcula la raíz cuadrada de un elemento.
square	Calcula el cuadrado de un elemento.
exp	Calcula el exponente de un elemento.
sign	Devuelve el signo de un elemento: 1(positivo), 0 (cero) o -1 (negativo).
isnan	Devuelve un arreglo booleano indicando si los valores están vacíos o no.

Manejo de Entradas/Salidas con Numpy

Numpy tiene la capacidad de importar y guardar elementos directamente en archivos externos. Esta utilidad es muy ventajosa cuando realizamos proyectos de ciencia de datos o de inteligencia artificial y necesitamos importar los conjuntos de datos con los que posteriormente vamos a desarrollar nuestros modelos o entrenar nuestros algoritmos.

Se muestra un ejemplo básico de la operación de salvar en un archivo, es importante destacar que, en caso de no especificarse, Numpy le asignara la extensión “npy” al fichero exportado:

```
arr5 = np.arange(20)
arr5
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19])
```

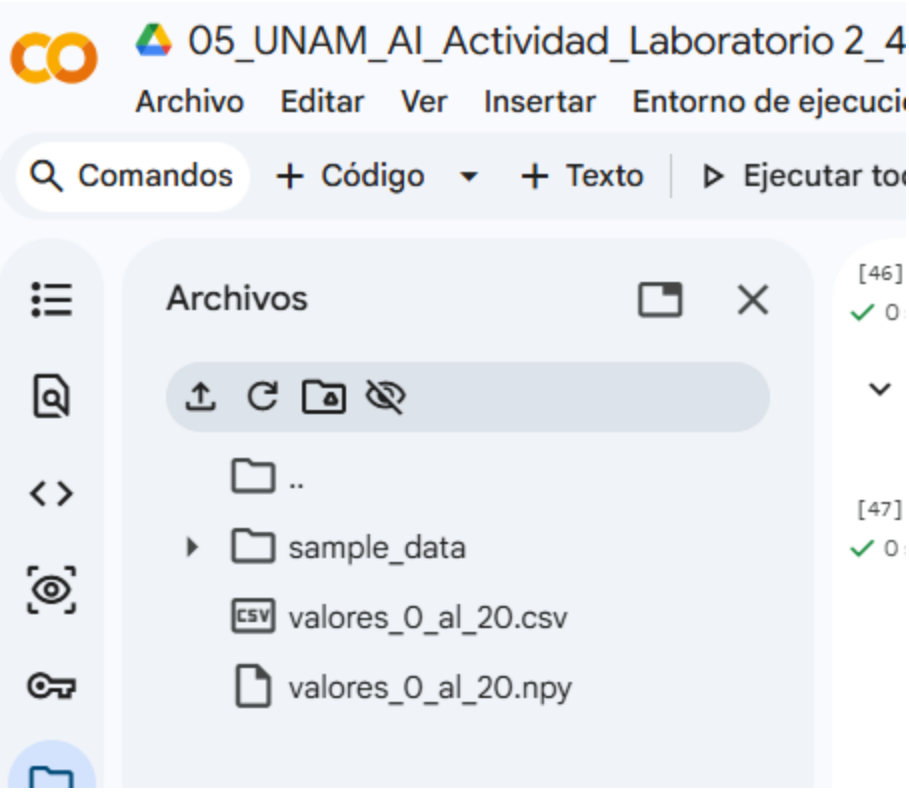
```
#Uso de la función save()
np.save("valores_0_al_20", arr5)
```

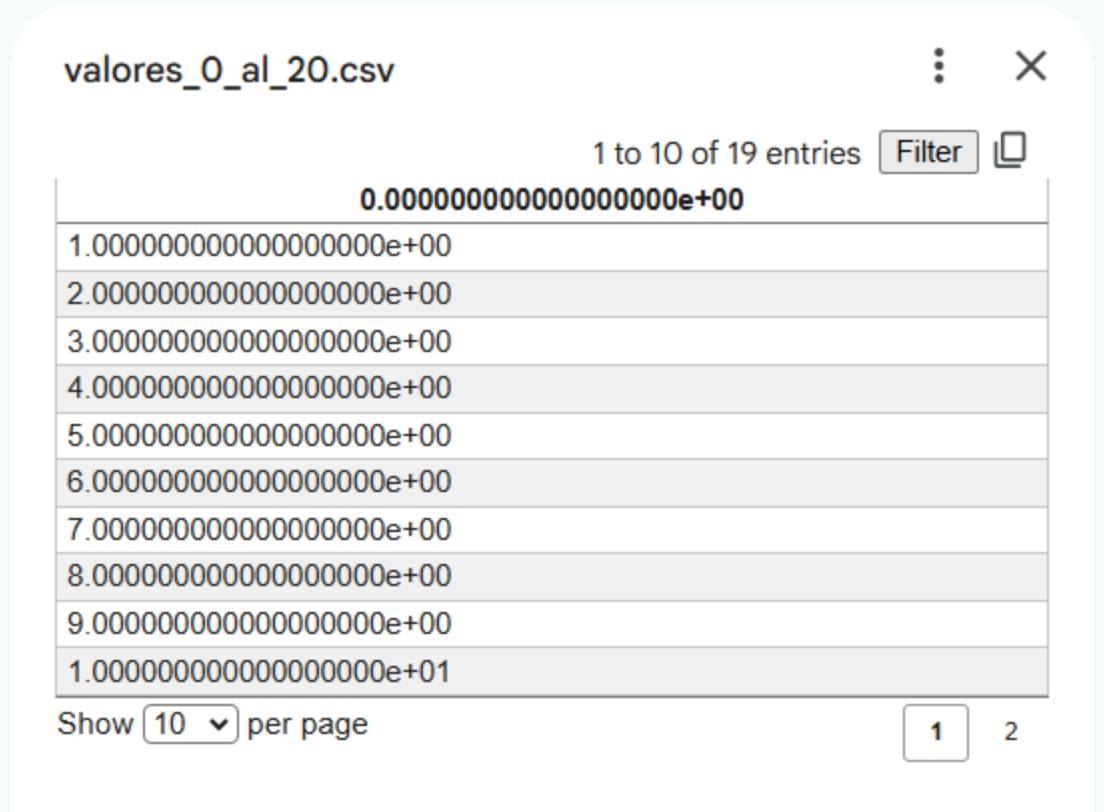
En caso de necesitar ampliar la compatibilidad con otros lenguajes numpy nos da la facilidad de exportar los datos como documentos de textos

```
arr5 = np.arange(20)
arr5
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19])
```

```
np.savetxt("valores_0_al_20.csv", arr5)
```





De igual forma podemos leer e importar desde nuestro almacenamiento interno los ficheros con la información antes generada

```
np.load("valores_0_al_20.npy")
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19])
```

Uso de librerías para manipulación de datos y análisis en Pandas

Otra de las librerías indispensables cuando desarrollamos proyectos de Ciencia de datos e inteligencia artificial es pandas. Pandas contiene una serie de herramientas diseñadas para limpiar y analizar estructuras de datos de un forma simple y rápida; mantiene una filosofía de trabajo orientada a arreglos por lo que se adapta muy bien a trabajar en conjunto con otras librerías que también utilizan este estilo. A pesar de que pandas comparte muchas características de numpy; la mayor diferencia es que pandas fue creada para trabajar con datos heterogéneos y tabulares, mientras que numpy, opera de forma preferencial con arreglos de datos homogéneos (McKinney, 2017).

Pandas es un proyecto de código libre desde 2010 y es mantenido por una amplia comunidad de más de 800 programadores, que a lo largo de todos estos años le han ido agregando mejoras para adaptarla a los nuevos retos del procesamiento de datos en el día a día.

La forma recomendada de incorporar el paquete pandas dentro de nuestros proyectos se muestra:

```
import pandas as pd
```

Es muy común encontrar en la bibliografía el alias "pd" asociado a la librería pandas. En el desarrollo de este tema esta será la forma que utilizaremos para referenciarla. En ocasiones solo necesitamos importar algunas funciones de la librería, las más comunes se muestran:

```
from pandas import Series, DataFrame
```

Introducción a las estructuras de datos en Pandas

Pandas opera con dos estructuras de datos básicas: series y dataframe. Estas estructuras no son obligatorias en todas las soluciones, pero tienen la facilidad de adaptarse a la mayoría de los casos de uso.

Series

Las series son arreglos unidimensionales que contienen una secuencia de valores (similares a los de numpy) asociados a otro arreglo llamado "index", el cual contiene las etiquetas de los datos almacenados. La forma más simple de generar una serie se puede ver a continuación, donde se utiliza como base una lista de números y no se especifica el valor de los índices.

```
import pandas as pd
mi_primera_serie = pd.Series([4, 6, 10, 3])
mi_primera_serie
```

	0
0	4
1	6
2	10
3	3

dtype: int64

La representación interactiva que se muestra indica en la parte izquierda los valores de los índices, mientras que ubica a la derecha la información de la serie. Una forma más completa de declarar este elemento puede ser:

```
import pandas as pd
mi_segunda_serie = pd.Series(["Alex", "Perez", 186, 84], index=["nombre", "apel
mi_segunda_serie
```

	0
nombre	Alex
apellido	Perez
altura	186
peso	84

dtype: object

Otra forma de comprender estos conceptos es visualizar una serie como un diccionario ordenado de tamaño ajustado. Este acercamiento es útil en diversas situaciones, por ejemplo, podemos crear una serie pasando como argumento un diccionario:

```
alumnos = {'Sistemas': 120, 'Derecho': 89, 'Periodismo': 50, 'Arquitectura': 80
alumnos_series = pd.Series(alumnos)
alumnos_series
```

	0
Sistemas	120
Derecho	89
Periodismo	50
Arquitectura	80

dtype: int64

Si en el ejemplo anterior utilizamos una lista como índice, podemos comprobar como la librería pandas ajusta los elementos de la serie para no dejar valores vacíos:

```
carreras = ["Derecho", "Sistemas", "Periodismo", "Arquitectura", "Diseño"]
alumnos_series = pd.Series(alumnos, index=carreras)
alumnos_series
```

	0
Derecho	89.0
Sistemas	120.0
Periodismo	50.0
Arquitectura	80.0
Diseño	NaN

dtype: float64

El término “NaN” (no es un número), es utilizado por pandas para referir que un valor determinado no fue encontrado.

Dataframes

Según McKinney(2017), un dataframe representa una tabla rectangular de datos, que contiene una colección ordenada de columnas, donde cada una puede contener un tipo diferente de datos (numérico, cadena, booleano, entre otras). Dentro del dataframe existen dos índices: fila y columna; se puede visualizar como un diccionario de series que comparten el mismo índice. De manera interna, los datos se almacenan como uno o más bloques bidimensionales en lugar de listas, diccionarios o alguna otra estructura de datos unidimensional.

Un dataframe se puede generar de la siguiente forma:

```
alumnos_inscritos = {'Campus': ['Las Torres', 'Hermosillo', 'Cancún'],
                     'Matricula': [1500, 850, 986],
                     'Año': [2021, 2021, 2021]}
inscritos_dataframe = pd.DataFrame(alumnos_inscritos)
inscritos_dataframe
```

	Campus	Matricula	Año
0	Las Torres	1500	2021
1	Hermosillo	850	2021
2	Cancún	986	2021

Próximos pasos: Generar código con `inscritos_dataframe` New interactive sheet

Podemos utilizar diferentes elementos como entradas del constructor de un dataframe, a continuación se muestran algunos de estos:

Tipo	Descripción
Ndarray	Matriz de datos, se puede incluir de firma opcional las etiquetas de las filas y columnas.
Diccionario de arreglos, listas o tuplas	Cualquiera de estas secuencias se convierte en una columna del dataframe, pero todas deben tener igual tamaño.
Diccionario de series	Cada valor se convierte en una columna, los índices de cada serie se unen para formar el índice de la fila resultante, si no se pasa un valor explícito.
Lista de diccionarios o series	Cada diccionario interno se convierte en una columna; las claves se unen para formar el índice de fila resultante, de igual manera al Diccionario de series.
Otro dataframe	Se puede utilizar otro dataframe como constructor, en este caso se mantienen los índices del dataframe constructor, si no se especifican otros.

Los dataframes poseen varios métodos y propiedades muy útiles, a continuación se definen algunos de estos:

Método	Descripción
append	Concatena los elementos y produce un nuevo índice.
diference	Calcula la diferencia entre conjuntos y las devuelve como un índice.
intersection	Devuelve la intersección de los conjuntos.
union	Devuelve la unión de los conjuntos.
isin	Devuelve un arreglo booleano indicando cuáles de los valores están dentro la de colección.
delete	Devuelve un nuevo dataframe sin el elemento indicado por el índice.
drop	Devuelve un nuevo dataframe sin el conjunto los elementos indicados.
insert	Devuelve un nuevo dataframe agregando el nuevo conjunto los elementos indicados

Operaciones con Dataframes

Cuando trabajamos con dataframes existen varias operaciones que podemos realizar, a continuación, veremos algunas de las más simples y útiles:

Obtener los tipos de datos de los campos:

```
inscritos_dataframe.dtypes
```

	0
Campus	object
Matricula	int64
Año	int64

dtype: object

Obtener las características de la secuencia inmutable utilizada para la indexación y alineación:

```
inscritos_dataframe.index
```

```
RangeIndex(start=0, stop=3, step=1)
```

Obtener las etiquetas de las columnas:

```
#Obtención de las etiquetas de las columnas.
inscritos_dataframe.columns
```

```
Index(['Campus', 'Matricula', 'Año'], dtype='object')
```

Realizar la transposición del dataframe:

```
inscritos_dataframe.T
```

	0	1	2
Campus	Las Torres	Hermosillo	Cancún
Matricula	1500	850	986
Año	2021	2021	2021

Ordenar elementos por valores o por índices:

```
#Ordenar elementos por valores.
inscritos_dataframe.sort_values(by=["Matricula"])
```

	Campus	Matricula	Año
1	Hermosillo	850	2021
2	Cancún	986	2021
0	Las Torres	1500	2021

```
inscritos_dataframe.sort_index(ascending=False)
```

	Campus	Matricula	Año
2	Cancún	986	2021
1	Hermosillo	850	2021
0	Las Torres	1500	2021

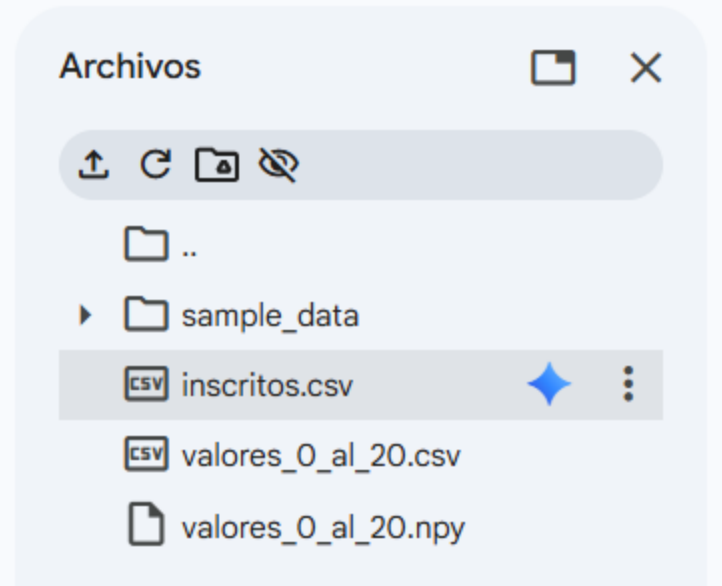
En pandas también podemos guardar nuestros datos en archivos externos. Uno de los formatos más populares para compartir información es archivo de valores delimitado por comas (csv); en las Tabla 38 y Tabla 39 se muestran ejemplos de cómo se pueden guardar y leer nuestros datos en este tipo de fichero:

```
#Guardar dataframe en un archivo csv.
inscritos_dataframe.to_csv('inscritos.csv')
```

```
otro_dataframe = pd.read_csv('inscritos.csv')
otro_dataframe
```

	Unnamed: 0	Campus	Matricula	Año
0	0	Las Torres	1500	2021
1	1	Hermosillo	850	2021
2	2	Cancún	986	2021

Próximos pasos: Generar código con otro_dataframe | New interactive sheet



En el desarrollo de este tema dimos un pequeño recorrido por dos principales librerías que posee Python para el manejo de datos. Tanto numpy como pandas, utilizan un paradigma enfocado a arreglos, lo cual les ofrece una verdadera ventaja al momento de procesar grandes cantidades de elementos. Al utilizar librerías escritas en bajo nivel, se abre la posibilidad de combinar nuestras aplicaciones con programas escritos en otros lenguajes, además de aprovechar las potencialidades del hardware donde se ejecutan a su máxima expresión.

En la mayoría de las implementaciones de ciencia de datos, aprendizaje automático o inteligencia artificial en general, veremos a numpy y a pandas trabajando en combinación con otras librerías, esa posibilidad de compatibilidad junto con la versatilidad y comodidad de escribir programas con la sintaxis de Python las convierten en el set de herramientas básico de cualquier científico o ingeniero que necesite operar con datos.

Python ha estado involucrado en varios de los super proyectos tecnológicos de la última década, sin duda estará presente en muchos más; en cualquier momento podemos recibir la noticia de otro resultado increíble y puede ser que seas tú el experto que está detrás del mismo.

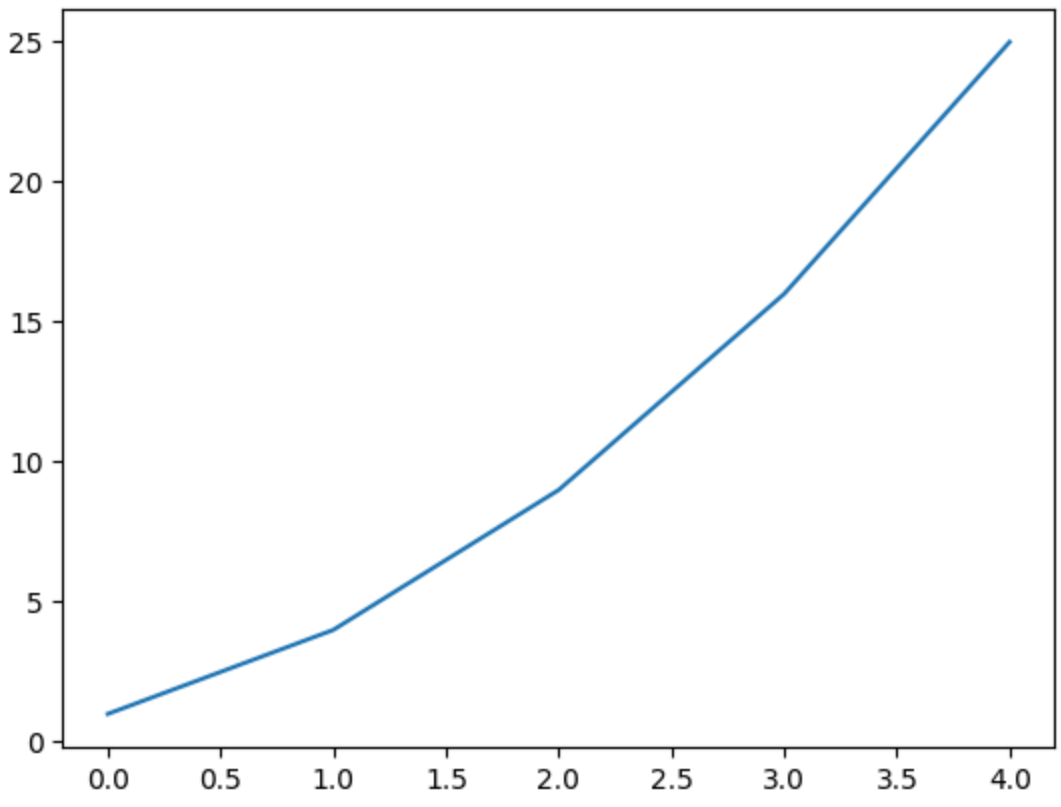
Librerías para graficado en Matplotlib

Con la librería Matplotlib se pueden generar gráficos como líneas y diagramas de dispersión de una forma muy simple. En los siguientes ejemplos aprenderemos a utilizarla dentro de nuestro programa para visualizar primeramente un pequeño conjunto

de datos definidos de forma manual y a continuación un conjunto más grande, generado automáticamente mediante una variable aleatoria.

Como primer ejemplo, Matthes (2019) propone trazar un gráfico de una línea simple empleando Matplotlib y después más adelante podrás personalizarlo para crear una visualización de datos con más información. Se utiliza la secuencia del valor de los números 1, 2, 3, 4 y 5 elevados al cuadrado como datos para el gráfico.

```
#Gráfico de una secuencia de números usando Matplotlib.
import matplotlib.pyplot as plt
cuadrados = [1, 4, 9, 16, 25]
fig, ax = plt.subplots()
ax.plot(cuadrados)
plt.show()
```



El primer paso fue importar el módulo pyplot usando el alias plt para no repetir toda la palabra, esa suele ser una práctica muy común al utilizarlo. El módulo contiene una serie de funciones que generan gráficos y diagramas.

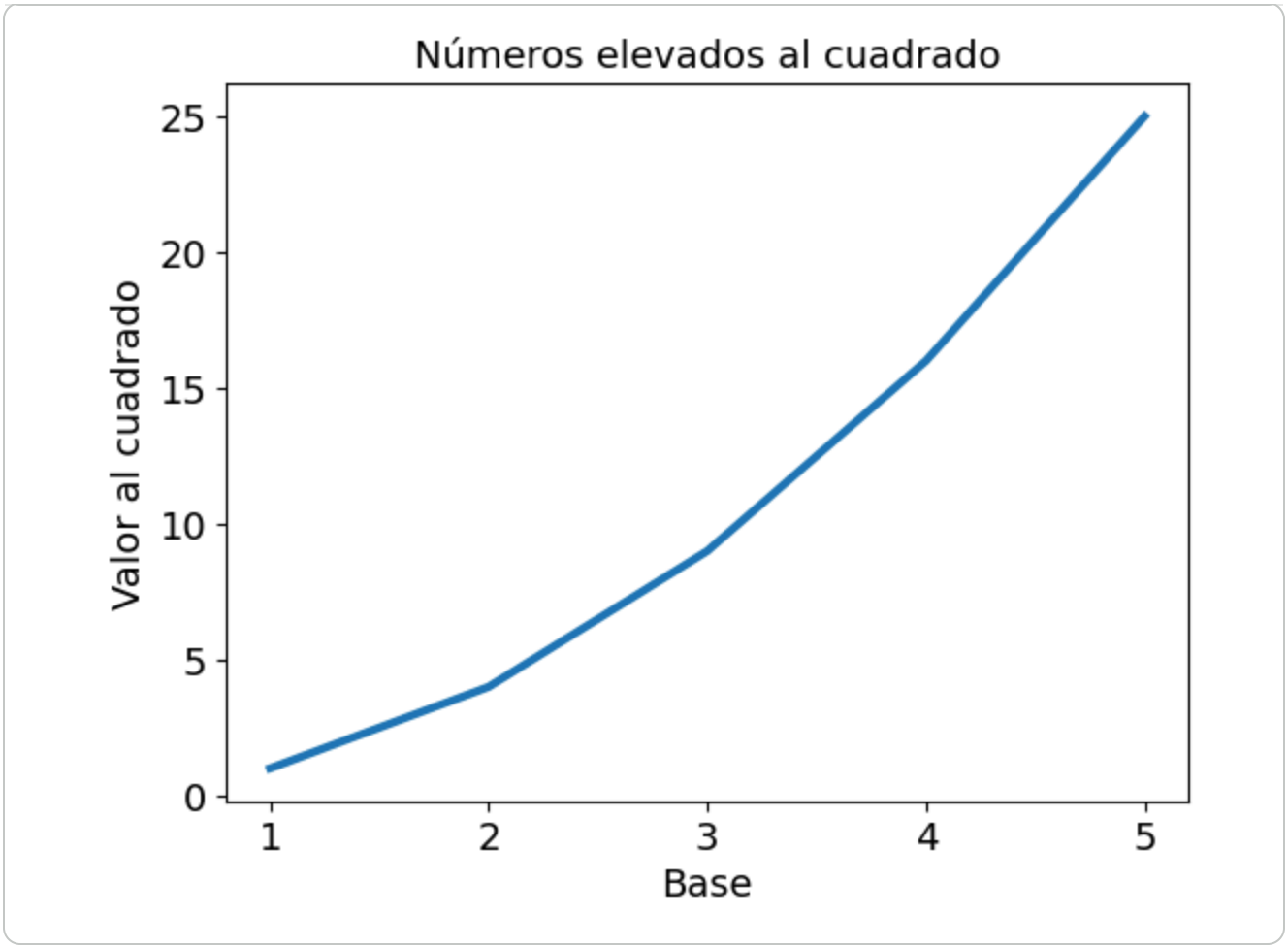
Después, se crea una lista llamada cuadrados para contener los datos que se trazarán y entonces en la línea 3 se llama a la función subplots() de Matplotlib. Esta función puede generar uno o más gráficos en la misma figura.

La variable fig representa la figura completa o la colección de gráficos que se generen. La variable ax, por su parte, representa un solo gráfico en la figura y se usará la mayor parte del tiempo.

Como últimos pasos para este ejemplo se usa el método plot(), cuya función es graficar los datos proporcionados, mientras que la función plt.show () abre el visor de Matplotlib y muestra el gráfico, como pudo apreciarse en el resultado de la Tabla 1. Sin embargo, al analizar el gráfico es que el valor de 4 en el eje horizontal no coincide con el valor de 25 en el eje vertical, por lo que en el siguiente ejemplo se ajustará y corregirá el gráfico agregando otra lista al programa llamada valores_entrada.

Aunque el gráfico anterior muestra el comportamiento de los datos, el tamaño de las etiquetas es demasiado pequeño y la línea un poco delgada para apreciarse fácilmente. Afortunadamente, Matplotlib permite ajustar cada característica del gráfico. A continuación, se usarán algunas de las personalizaciones disponibles para mejorar la legibilidad a través del siguiente código:

```
#Gráfico de una secuencia de números ajustando parámetros.
import matplotlib.pyplot as plt
valores_entrada = [1, 2, 3, 4, 5]
cuadrados = [1, 4, 9, 16, 25]
fig, ax = plt.subplots()
ax.plot(valores_entrada, cuadrados, linewidth=3)
#Establecer el título del gráfico y los ejes de la etiqueta.
ax.set_title("Números elevados al cuadrado", fontsize=14)
ax.set_xlabel("Base", fontsize=14)
ax.set_ylabel("Valor al cuadrado", fontsize=14)
#Establecer el tamaño de las etiquetas de las marcas.
ax.tick_params(axis='both', labelsize=14)
plt.show()
```



El parámetro linewidth utilizado en la línea 5 del código controla el grosor de la línea del gráfico generada por la función plot(); también se incluye a la lista valores_entrada como parámetro, declarada en la línea 2 del código, con el propósito de ajustar los valores del eje horizontal y coincidan con relación a los del eje vertical. El método set_title () en la línea 7 establece un título para el gráfico. Los parámetros de tamaño de fuente, que aparecen repetidamente en el código, controlan el tamaño del texto en los elementos del gráfico.

Por su parte, los métodos set_xlabel() y set_ylabel() declarados en la línea 8 permiten establecer un título para cada uno de los ejes x y y, y el método tick_params() de la línea 11 estiliza las marcas de graduación, sus argumentos afectan las marcas de graduación eje x y y (axis='both') y establece el tamaño de fuente de las etiquetas en 14 (labelsize=14).

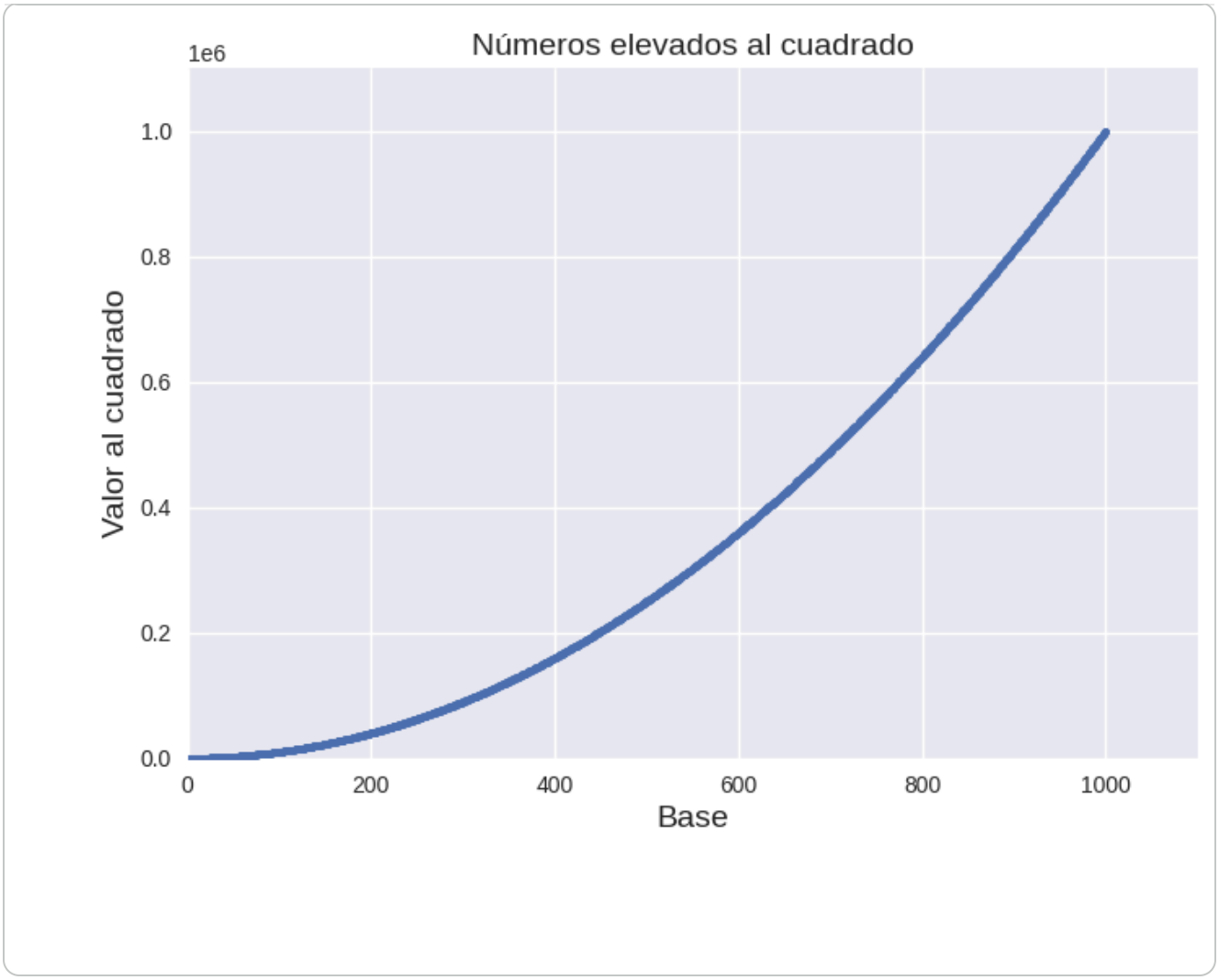
Como puedes ver a continaución, resulta mucho más fácil leer el gráfico en comparación con el generado por el código de la gráfica anterior, ya que el tipo de etiqueta es más grande y el gráfico de líneas es más grueso. A menudo, vale la pena experimentar con los diferentes parámetros para encontrar cual es el mejor gráfico resultante.

Sin embargo, escribir listas a mano puede resultar ineficaz, sobre todo cuando se tienen muchos puntos. Por lo que ahora, en vez de pasar los puntos a una lista, se usará un bucle para que Python los genere automáticamente.

Si se desea guardar el gráfico en un archivo se puede hacer por medio de la función plt.savefig().

El siguiente ejemplo sirve para 1000 puntos:

```
#Gráfico de una secuencia de números generada automáticamente.
import matplotlib.pyplot as plt
x_valores = range(1, 1001)
y_valores = [x**2 for x in x_valores]
plt.style.use('seaborn-v0_8')
#prueba con este otro estilo y usa el que más te guste
#plt.style.use('ggplot')
fig, ax = plt.subplots()
ax.scatter(x_valores, y_valores, s=10)
#Establecer el título del gráfico y los ejes de la etiqueta.
ax.set_title("Números elevados al cuadrado", fontsize=14)
ax.set_xlabel("Base", fontsize=14)
ax.set_ylabel("Valor al cuadrado", fontsize=14)
#Establezca el rango para cada eje.
ax.axis([0, 1100, 0, 1100000])
plt.savefig('grafico_cuadrados.png', bbox_inches='tight')
plt.show()
```




```
plt.show()
```

```
#Para ver más estilos y elegir el que más te guste
import matplotlib.pyplot as plt
print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid'
```

En la línea 2 se establece un rango de valores de x que contienen los números del 1 al 1000. Después, se declara una lista que genera los valores de y recorriendo los valores de x (para x en valores_x), elevando al cuadrado cada número (x**2) y que almacenará los resultados en y_values. Lo siguiente es pasar las listas de entrada y salida a la función scatter() en la línea 6 del código. Debido a que esto representa un conjunto de datos grande se utiliza un tamaño de punto más pequeño.

En la línea 12 se usa el método axis() para especificar el rango de cada eje. Este método requiere cuatro valores: los valores mínimo y máximo para el eje x y y. Es así como se

genera el resultado mostrado en la Tabla 3, donde el eje x va de 0 a 1100 y el eje y de 0 a 1,100,000.

El primer argumento es el nombre de archivo para la imagen del gráfico que se guarda en la misma carpeta que el programa. El segundo argumento recorta los espacios en blanco adicionales de la trama. Si desea el espacio en blanco adicional alrededor de la trama, simplemente omita este argumento.

Librerías estadísticas en Seaborn y Plotly

Seaborn (s.f.) es una biblioteca para hacer gráficos estadísticos en Python basada en Matplotlib y se integra muy de cerca con las estructuras de datos de pandas. Es una herramienta que ayuda a explorar y comprender datos. Sus funciones de trazado operan con matrices que contienen conjuntos de datos completos y realizan internamente un mapeo semántico y estadístico que permite producir gráficos que relevan información importante. Su API declarativa orientada a conjuntos de datos le permite centrarse en lo que significan los diferentes elementos de los gráficos en vez de los detalles sobre como dibujarlos.

Para los siguientes ejemplos se ha creado un cuaderno nuevo en Jupyter Notebook bajo el nombre Uso de Seaborn y Plotly como se muestra a continuación:

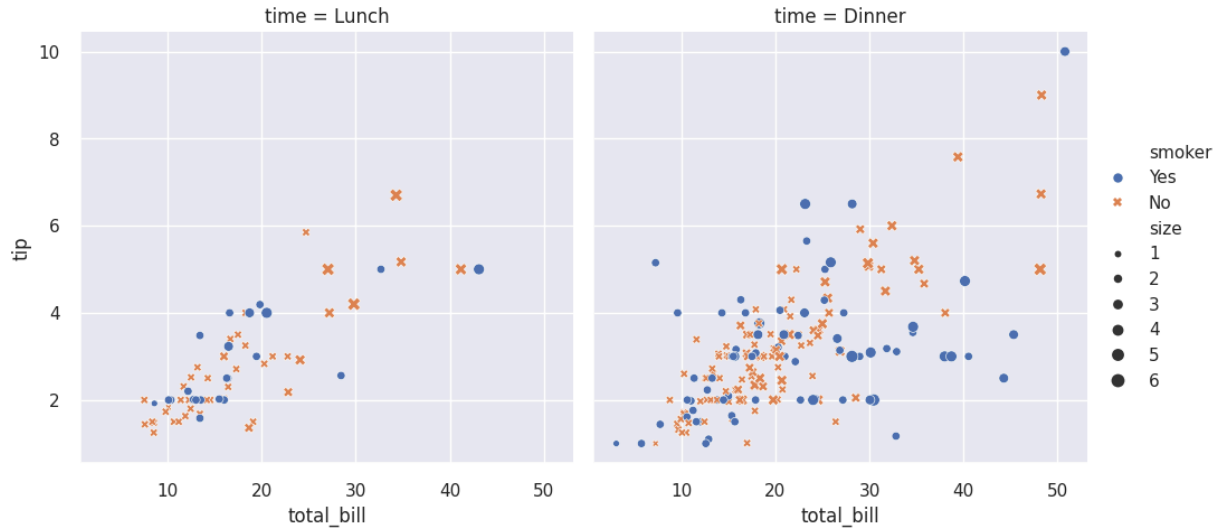
El siguiente ejemplo te permitirá explorar algunas características y funciones de Seaborn:

```
#Gráfica de un conjunto de datos predeterminado usando Seaborn.
#Se importa seaborn
import seaborn as sns
#Se aplica el tema predeterminado
sns.set_theme()
#Se carga una base de datos de ejemplo
propinas = sns.load_dataset("tips")
#Se realiza la visualización de los datos
sns.relplot(

data=propinas,
x="total_bill", y="tip", col="time",
hue="smoker", style="smoker", size="size",

)
```

```
<seaborn.axisgrid.FacetGrid at 0x7b5341e4ea50>
```



En la línea 2 se importa la librería Seaborn como la abreviatura sns. Como ya se mencionó anteriormente, Seaborn usa Matplotlib para trazar sus gráficos. Al utilizar Seaborn en Jupyter ya no es necesario llamar a la función matplotlib.pyplot.show ().

Al declarar sns.set_theme() en la línea 4 se emplea el sistema de matplotlib rcParam, lo que significa que el aspecto de cada gráfico es modificado. Más allá del tema predeterminado, hay otras opciones y el estilo se puede controlar de forma independiente. Si se prefiere trabajar con los valores predeterminados de Matplotlib o con un tema diferente este paso puede omitirse.

La función load_dataset() usada en la línea 6 permite el acceso rápido a un conjunto de datos de ejemplo. El conjunto de datos está hecho en una estructura de Pandas (s.f), la cual a su vez es una herramienta de manipulación y análisis de datos de código abierto, también construida en Python.

El código escrito a partir de la línea 7 sirve para llevar a cabo la gráfica, la cual es una relación entre cinco variables en el conjunto de datos tips (propinas) haciendo una sola llamada a la función seaborn relplot(). Para esto, se proporcionan únicamente los nombres de las variables y se especifica el rol de cada una de ellas. A diferencia de

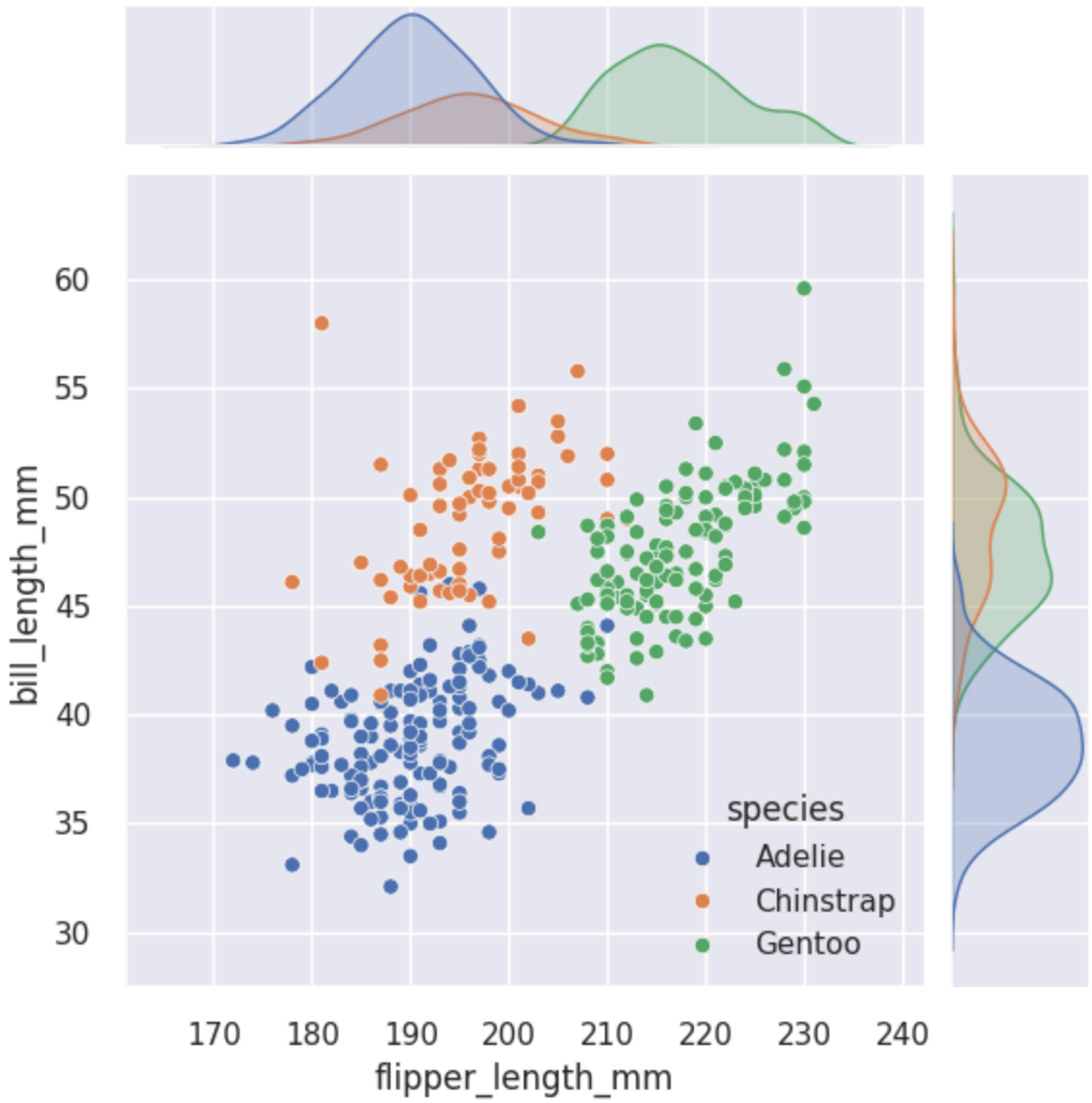
cuando se usa Matplotlib de forma directa, no es necesario especificar los atributos de los elementos en términos de valores de color o códigos de marcador.

Detrás del código del programa, Seaborn traduce los argumentos que Matplotlib entiende. Este enfoque permite concentrarse en el análisis de los datos en lugar de los detalles de cómo controlar Matplotlib.

Otra aplicación de Seaborn es la combinación de varios tipos de gráficos para proporcionar resúmenes informativos de un conjunto de datos de forma rápida y práctica. En el siguiente ejemplo se hace uso de la función jointplot() para trazar la distribución conjunta entre dos variables junto con la distribución marginal de cada variable:

```
#Combinación de gráficas para un conjunto de datos predeterminado en Seaborn.
pinguinos = sns.load_dataset("penguins")
sns.jointplot(data=pinguinos, x="flipper_length_mm", y="bill_length_mm", hue="s
```

```
<seaborn.axisgrid.JointGrid at 0x7b5341e4fb60>
```



Por otra parte, Matthes (2019) explica que el paquete de Plotly es una herramienta útil para producir visualizaciones interactivas, en especial cuando se crean visualizaciones que deben mostrarse en un navegador y es necesario escalarlas de manera automática para que se adapten a la pantalla del espectador.

Las visualizaciones que genera Plotly también son interactivas, cuando el usuario pasa el cursor sobre ciertos elementos en la pantalla, la información sobre ese elemento se resalta.

A continuación, se analizarán los resultados de tirar un dado. Cuando se lanza un dado normal de seis caras, cualquiera de los números del 1 al 6 tienen la misma probabilidad de salir. Entonces, se representará dicha probabilidad por medio de un conjunto de datos. Finalmente, se trazará un histograma usando Plotly.

Para hacer uso de Plotly en Python es necesario instalarlo previamente.

Una vez descargado Plotly, puedes comenzar realizando el siguiente programa para crear una clase llamada Dado, con la cual se puede crear un dado de seis lados y lanzarlo por medio del método lanzar():

```
#Declaración de la clase Dado.
from random import randint
class Dado:
  """Clase que representa a un dado."""
  def __init__(self, num_lados=6):
    """Se consideran a un dado con 6 lados."""
    self.num_lados = num_lados
  def lanzar(self):
    """"Devuelve un valor aleatorio entre 1 y el número de lados."""
    return randint(1, self.num_lados)
```

Antes de crear una visualización basada en la clase declarada anteriormente, se crea el objeto dado y se imprimen los resultados para verificar que los resultados sean razonables:

```
#Se crea un dado de seis lados
#Simulación de múltiples lanzamientos de un dado.
dado = Dado()
#Se realizan algunos lanzamientos y se guardan en una lista.
resultados = []
for num_lanzamiento in range(100):
  resultado = dado.lanzar()
  resultados.append(resultado)
print(resultados)
```

```
[4, 1, 3, 5, 2, 5, 5, 2, 4, 2, 4, 6, 1, 4, 4, 4, 4, 1, 5, 3, 4, 1, 4, 1, 5, 2, 1
```

Un análisis rápido de los resultados anteriores permite ver que la clase Dado funciona. Se aprecian valores entre 1 y 6, por lo que la función está devolviendo los valores más pequeños y más grandes posibles, y como no sale ningún 0, 7 u otra cosa, se ha comprobado que todos los resultados están en el rango apropiado.

Mediante el último ejemplo se determinará cuántas veces aparece cada número exactamente. Con una lista de frecuencias, es posible hacer un histograma de los resultados, el cual es un gráfico de barras que muestra la frecuencia con la que ocurren ciertos resultados. El código para crearlo sería el siguiente:

```
from plotly.graph_objs import Bar, Layout
from plotly import offline
#Análisis de los resultados.
```

```
frecuencias = []
for valor in range(1, dado.num_lados+1):
  frecuencia = resultados.count(valor)
  frecuencias.append(frecuencia)

#Visualización de los resultados
x_valores = list(range(1, dado.num_lados+1))
datos = [Bar(x=x_valores, y=frecuencias)]

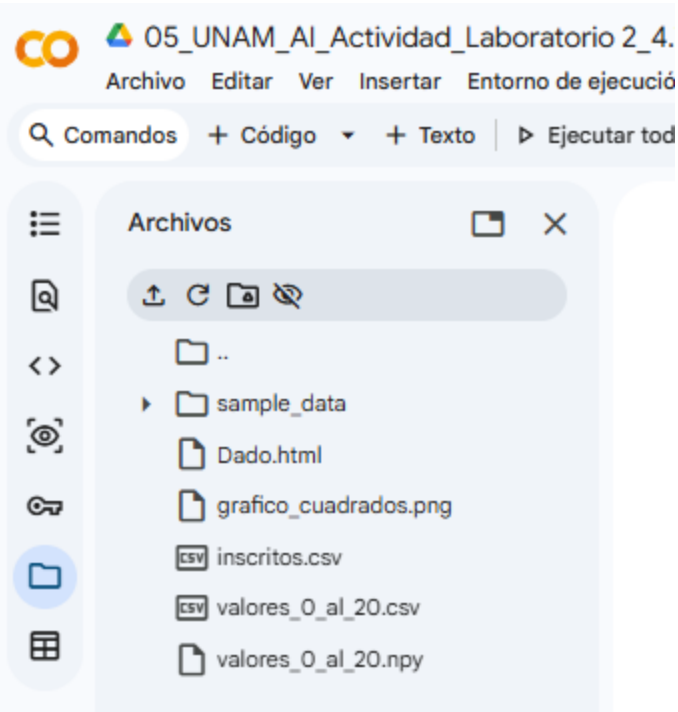
x_eje_config = {'title': 'Resultado'}
y_eje_config = {'title': 'Frecuencia del resultado'}
my_layout = Layout(title='Resultados de lanzar el dado 1000 veces',

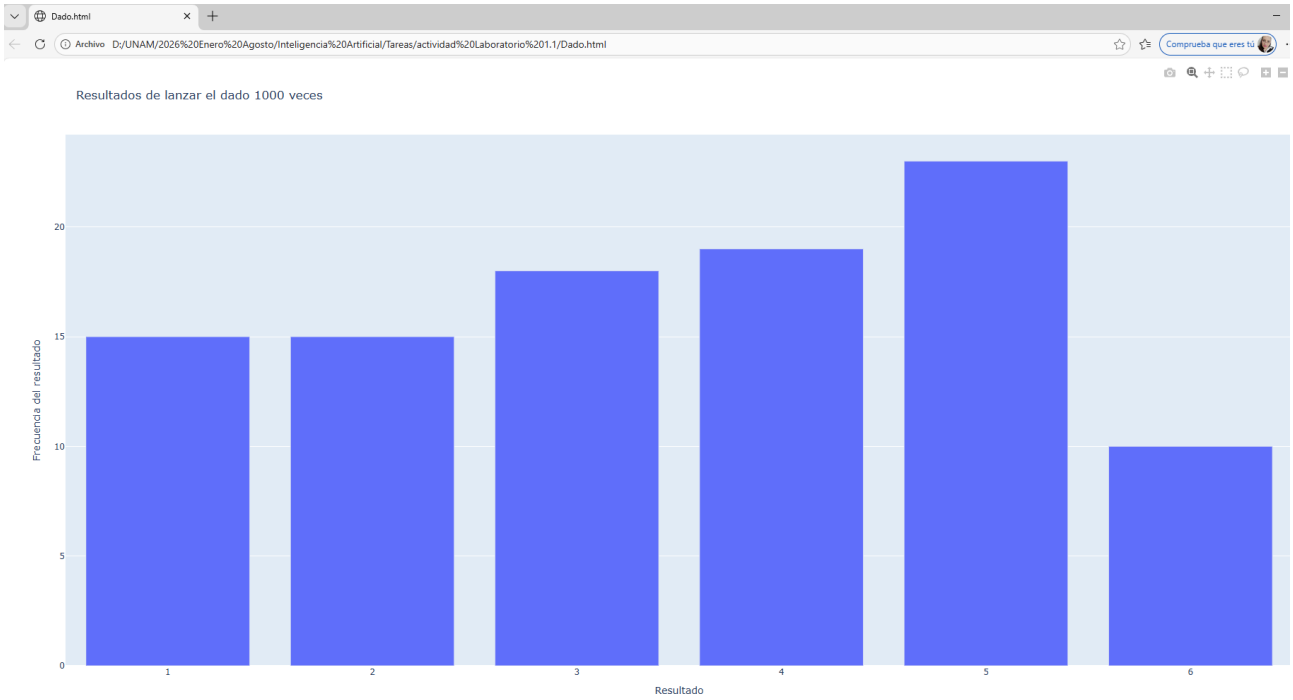
xaxis=x_eje_config, yaxis=y_eje_config)

offline.plot({'data': datos, 'layout': my_layout}, filename='Dado.html')
```

```
'Dado.html'
```

Cuando se ejecuta el programa probablemente se abra una pestaña en tu navegador mostrando el archivo Dado.html. Si eso no llegara a suceder automáticamente, abre una nueva pestaña en cualquier navegador web y escribe la ruta donde hayas guardado a Dado.html. En cualquiera de los casos, debería verse un gráfico como el mostrado





Desafio

Revisa:

- Los conceptos generales de los lenguajes de programación y sus enfoques hacia a la inteligencia artificial.
- El funcionamiento de los diccionarios y conjuntos en Python.
- La forma de declarar funciones y combinarlas con las estructuras de control en Python.
- La instalación y uso de las librerías NumPy, Pandas y Matplotlib en Python.

Usa la información que se descarga de Kaggle. Global Air Quality Dataset 🌎

Realiza las siguientes operaciones:

- Importa las librerías NumPy, Pandas y Matplotlib a tu notebook.
- Genera un Dataframe con los datos incluidos en el fichero de datos obtenido de Kaggle.
- Organiza la información dentro de Dataframe de la forma que mejor se adecue a la problemática que se desea analizar, modifica el tipo de variable en caso de ser necesario.
- Selecciona una muestra de los datos.
- Aplica una función matemática que te permita sintetizar o analizar parte de los datos dentro del Dataframe.

- Representa gráficamente la información que aportan.

```
import pandas as pd
import os
import kagglehub

# Descargar información de Kaggle
path = kagglehub.dataset_download("waqi786/global-air-quality-dataset")

print("Path to dataset files:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/waqi786/global-air-quality-datase
100%|██████████| 258k/258k [00:00<00:00, 490kB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/waqi786/global-air-qualit
```

```
#Listar los archivos en la ruta que descargaste
archivos = os.listdir(path)
print("Archivos encontrados:", archivos)

#Supongamos que el archivo se llama 'global_air_quality_dataset.csv'
#Creamos la ruta completa combinando el path con el nombre del archivo
csv_path = os.path.join(path, archivos[0]) # archivos[0] toma el primer archivc

#Cargamos el DataFrame (¡Recuerda la D y F mayúsculas!)
df = pd.read_csv(csv_path)

#Ver los primeros datos
df.head()
```

```
Archivos encontrados: ['global_air_quality_data_10000.csv']
```

	City	Country	Date	PM2.5	PM10	NO2	SO2	CO	O3	Temperature
0	Bangkok	Thailand	2023-03-19	86.57	25.19	99.88	30.63	4.46	36.29	17.67
1	Istanbul	Turkey	2023-02-16	50.63	97.39	48.14	8.71	3.40	144.16	3.46
2	Rio de Janeiro	Brazil	2023-11-13	130.21	57.22	98.51	9.92	0.12	179.31	25.29

Próximos pasos: Generar código con `df` New interactive sheet

- Organizar la información: Filtra las columnas que realmente vas a usar para que el DataFrame sea legible.

- Función Matemática: Vamos a crear un índice de "Impacto de Salud" sumando contaminantes o convirtiendo unidades.
- Representación Gráfica: Usaremos Matplotlib con las correcciones de estilo y guardado que vimos antes.

```
# Filtramos donde la columna 'Country' sea exactamente 'Mexico'
# Nota: Revisa si en tu archivo viene como 'Mexico' o 'Mexico ' (a veces traen
df_mexico = df[df['Country'] == 'Mexico']

# Verificamos cuántos datos tenemos de México
print(f"Registros encontrados para México: {len(df_mexico)}")
df_mexico.head()
```

```
Registros encontrados para México: 463
```

	City	Country	Date	PM2.5	PM10	NO2	SO2	CO	O3	Temperature
17	Mexico City	Mexico	2023-01-26	82.40	196.86	20.76	47.75	9.57	21.74	29.97
32	Mexico City	Mexico	2023-05-08	120.90	33.70	97.53	9.28	5.67	191.24	22.76
41	Mexico City	Mexico	2023-08-18	51.47	137.97	97.83	3.45	7.12	194.18	-3.68

```
df_mexico.describe()
```

	PM2.5	PM10	NO2	SO2	CO	O3	Temp
count	463.000000	463.000000	463.000000	463.000000	463.000000	463.000000	46
mean	78.855054	104.526868	50.863650	26.022138	5.141339	105.686285	1
std	41.168497	54.309229	26.982939	13.990531	2.896417	55.658559	1
min	5.420000	10.790000	5.200000	1.050000	0.100000	10.310000	-
25%	43.755000	59.450000	27.670000	13.470000	2.555000	56.635000	
50%	77.620000	102.090000	50.490000	26.890000	5.280000	107.310000	1
75%	114.280000	151.050000	74.880000	37.880000	7.695000	151.050000	2
max	149.220000	199.890000	100.000000	49.950000	9.950000	198.900000	3

PROPUESTA DE SOLUCIÓN